# CS-433 Machine learning: project 1

Erdogan Ergit, Hugo Dutheil, Mariya Rakytyanska

## 1 Introduction

For this project, we decided to implement a PCA since this method is often well adapted for classification problems such as what was given here. PCA consists of building a set principle components for each data entry in the training set, find the components of a the test set, and find the best matching components from the training set to match the output.

## 2 Pipeline Architecture and Stages

The pipeline consists of four key stages, moving from raw data preparation to final predictions:

1. **Data Cleaning and Filtering**
   - *Missing Value Thresholding:* The script checks both columns (features) and rows (samples/people) against a 75% threshold. Any column or row containing too many missing (NaN) values is discarded to maintain data integrity.
   - *Imputation:* Remaining missing values in the feature matrix are replaced with zeros using 'np.nan_to_num' to prevent numerical errors during matrix multiplication and singular value decomposition.
2. **Feature Scaling and Correlation Selection**
   - *Standardization:* Features are centered and scaled to have a mean of 0 and a standard deviation of 1, ensuring all variables contribute equally regardless of their original units.
   - *Correlation Filter:* Absolute correlations between each feature and the target variable $y$ are computed. Only the top 30% of features ($70^{\text{th}}$ percentile and above) are retained to filter out noise.
3. **Dimensionality Reduction via PCA**
   - *Singular Value Decomposition (SVD):* PCA is implemented using SVD to transform the high-dimensional feature space into an orthogonal coordinate system (principal components).
   - *Component Selection:* The pipeline selects the top $k = 15$ principal components ('scores_train_k' and 'scores_test_k') to capture the primary variance of the dataset while ignoring minor noise.
   - *Test Set Projection:* The test dataset is scaled using the training set's mean and standard deviation, then projected into the exact same PCA coordinate space using the training rotation matrix ($\mathbf{V}^T$).
4. **Batch Processing and Nearest-Neighbor Prediction**
   - *Memory-Efficient Batching:* To avoid memory overflow when computing pairwise distances between large training and test sets, the test set is split into batches of size 1000.

- *Vectorized Euclidean Distance:* Distances are calculated efficiently using the expansion formula:
$$\|\mathbf{v} - \mathbf{w}\|^2 = \|\mathbf{v}\|^2 + \|\mathbf{w}\|^2 - 2\langle \mathbf{v}, \mathbf{w} \rangle$$
- *Prediction Mapping:* For each test instance, the index of the closest training sample is found using 'np.argmin', and its corresponding target value ($y$) is assigned as the prediction.

# 3 Summary of Parameters

| **Parameter** | **Value / Choice** |
|---|---|
| Missing Data Threshold | 75% (0.75) |
| Feature Selection Percentile | Top 30% ($70^{\text{th}}$ percentile cutoff) |
| Dimensionality ($k$) | 15 principal components |
| Batch Size | 1000 samples per iteration |
| Prediction Method | 1-Nearest Neighbor in PCA space |